

HRMS

Identity / User / Role / Permission

Users, Roles, Permissions, assignments, passwords and database-driven permission authorization

Technical implementation reference

1. Scope

This document covers Users, Roles, Permissions, UserRole and RolePermission assignments, password operations and database-driven permission authorization.

2. Security Architecture

```
Login
↓
JWT Token
↓
Authentication
↓
IUserContext (userId / employeeId / userName)
↓
Authorization Policy
↓
PermissionRequirement
↓
PermissionAuthorizationHandler
↓
UserRoles → Roles → RolePermissions → Permissions
↓
Controller
```

3. SQL Table Information

SQL table	Purpose	Important relationships
Users	Application login identity and user profile	UserRoles; employee/user association
Roles	Named security roles	RolePermissions
Permissions	Named capabilities	Referenced by RolePermissions
UserRoles	User-to-role assignment	UserId → Users; RoleId → Roles
RolePermissions	Role-to-permission assignment	RoleId → Roles; PermissionId → Permissions

Auditable assignment tables

UserRoles and RolePermissions were migrated to use an Id primary key plus the standard audit/soft-delete fields. Their business uniqueness is enforced separately so soft deletion can coexist with historical assignment rows.

Field	Typical SQL type	Purpose
Id	uniqueidentifier	Primary key
CreatedBy	uniqueidentifier NULL	User that created the row
CreatedOn	datetimeoffset	Creation timestamp
ModifiedBy	uniqueidentifier NULL	Last modifying user
ModifiedOn	datetimeoffset NULL	Last modification timestamp
DeletedBy	uniqueidentifier NULL	User that soft-deleted the row
DeletedOn	datetimeoffset NULL	Soft-delete timestamp
IsDeleted	bit	Soft-delete flag

Assignment uniqueness

```
UserRoles:
UNIQUE (UserId, RoleId) WHERE IsDeleted = 0

RolePermissions:
UNIQUE (RoleId, PermissionId) WHERE IsDeleted = 0
```

4. API Contract

Area	HTTP	Endpoint	Purpose
Users	GET	/api/User	List users
Users	GET	/api/User/{id}	Get user
Users	POST	/api/User	Create user
Users	PUT	/api/User/{id}	Update user
Users	POST	/api/User/{id}/deactivate	Deactivate user
Password	POST	/api/User/{id}/change-password	Change password
Password	POST	/api/User/{id}/reset-password	Administrative reset
Auth	POST	/api/Auth/login	Issue JWT
Auth	GET	/api/Auth/me	Read current authenticated user context
UserRole	POST/GET/DELETE	/api/UserRoles...	Assign/read/remove roles
RolePermission	POST/GET/DELETE	/api/RolePermissions...	Assign/read/remove permissions

Login

```
POST /api/Auth/login
```

```
{
  "userName": "anzar.admin",
  "password": "*****"
}
```

```
200 OK
```

```
{
  "token": ""
}
```

Reset password

```
POST /api/User/{id}/reset-password
Authorization: Bearer
```

```
{
  "userId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "newPassword": "Password@123"
}
```

```
204 No Content
```

Change password

```
POST /api/User/{id}/change-password
```

```
{
  "userId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "currentPassword": "*****",
  "newPassword": "Password@123"
}
```

```
204 No Content
```

5. Permission-Based Authorization

Policies are registered in `AuthorizationConfiguration`. The policy name and `PermissionRequirement` value are identical, for example `Employee.View` or `User.ResetPassword`.

```
options.AddPolicy("Employee.View", policy =>
  policy.Requirements.Add(
    new PermissionRequirement("Employee.View")));
```

```
[Authorize(Policy = "Employee.View")]
```

Handler decision logic

The `PermissionAuthorizationHandler` first requires an authenticated user ID. It loads the `User` and `UserRoles`, resolves active `Roles` and their `RolePermissions`, resolves active `Permissions`, and succeeds the requirement only when a permission name exactly matches the requested requirement.

6. Current Permission Examples

Permission	Protected operation
Employee.View	Read employee information
Employee.Create	Create employee
Employee.Update	Update employee
Employee.Delete	Delete employee
User.ResetPassword	Administrative password reset
Department.View	Read department information
Department.Create	Create department
Department.Update	Update department
Department.Delete	Delete department

7. Business and Security Rules

1. Authentication answers who the caller is; authorization answers what the caller may do.
2. Role membership alone must not grant an operation unless the required permission is present.
3. Inactive roles and inactive permissions do not authorize requests.
4. Soft-deleted `UserRole/RolePermission` rows must not participate in normal authorization.
5. Password hashes must never be returned by user APIs.
6. Change password and reset password are distinct use cases. Change password verifies the current password; reset password is an administrative operation protected by `User.ResetPassword`.
7. JWTs should be short-lived and signed with a securely stored secret key.
8. A newly changed role/permission assignment normally requires a fresh JWT only if authorization claims themselves are cached in the token; the current database-driven handler resolves assignments from the database.

8. Error Semantics

401 Unauthorized means authentication is missing or invalid. 403 Forbidden means the caller is authenticated but the required permission is not satisfied. 400 Bad Request is appropriate for malformed command/route mismatches. 404 Not Found is appropriate when the target user/role/permission does not exist.

9. Developer Change Guide

To add a new permission: add the permission record/seed data, register the policy in `AuthorizationConfiguration`, protect the controller action with `[Authorize(Policy = "Permission.Name")]`, ensure the `PermissionAuthorizationHandler` can resolve it through `RolePermissions`, and add tests for authorized and forbidden users.

Registration

```
builder.Services.AddIdentityAuthorization();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(...);

app.UseAuthentication();
app.UseAuthorization();
```

10. Database Migration Discipline

UserRoles and RolePermissions are existing assignment tables, so migrations must preserve existing assignments. When adding a primary key to populated tables, existing rows must receive unique IDs before the PK is created. Always inspect generated migration SQL and run model-change checks.

```
dotnet ef migrations has-pending-model-changes \  
--project "..\src\HRMS.Persistence" \  
--startup-project "..\src\HRMS.API"
```

11. Verification Checklist

Verify login and JWT validation; Auth/me; role assignment; permission assignment; Employee.View returns 200 for an authorized user; missing JWT returns 401; authenticated user without the permission returns 403; User.ResetPassword is denied to users without the permission and succeeds for an appropriately authorized user.